Claro. Como o código será refeito do zero e o **banco de dados é a primeira etapa**, esta versão deixa o documento independente da implementação antiga em Java e já considera a nova stack: **SQL Server, Prisma, Express.js, TypeScript e Docker**.

Também ajustei os requisitos para que eles sirvam diretamente de base para a definição das entidades, relacionamentos e regras do banco.

DOCUMENTO DE REQUISITOS DE SOFTWARE

SISTEMA DE GESTÃO DE PEDIDOS – CHIC DOG

1 INTRODUÇÃO

1.1 Propósito

Este documento tem como objetivo especificar os requisitos do Sistema de Gestão de Pedidos da empresa Chic Dog, estabelecendo as principais funcionalidades, regras de negócio, requisitos não funcionais, usuários envolvidos e informações que deverão ser gerenciadas pelo sistema.

A documentação servirá como base para as etapas de modelagem e desenvolvimento do sistema, especialmente para a construção do banco de dados e, posteriormente, para a implementação da API e das demais funcionalidades da aplicação.

1.2 Escopo

O Sistema de Gestão de Pedidos da Chic Dog terá como finalidade auxiliar no gerenciamento do atendimento, registro de pedidos, preparação dos produtos e controle dos pagamentos realizados pelos clientes.

O sistema deverá permitir que funcionários autenticados realizem operações de acordo com seus respectivos perfis de acesso. O garçom será responsável principalmente pelo registro e gerenciamento dos pedidos, o cozinheiro pelo acompanhamento e atualização dos pedidos em preparação e o gerente pelo gerenciamento e acompanhamento das operações do estabelecimento.

O sistema deverá armazenar informações referentes aos usuários, pedidos, itens dos pedidos e pagamentos, mantendo os relacionamentos entre essas informações de forma consistente.

O backend será desenvolvido utilizando TypeScript e Express.js, com Prisma como ORM para acesso ao banco de dados SQL Server. O ambiente de desenvolvimento e execução utilizará Docker para facilitar a configuração e padronização dos serviços.

1.3 Objetivos do Sistema

Os principais objetivos do sistema são:

- Organizar o processo de registro dos pedidos;
- Reduzir erros no registro e processamento dos pedidos;
- Facilitar a comunicação entre atendimento e cozinha;
- Permitir o acompanhamento do status dos pedidos;
- Registrar os pagamentos relacionados aos pedidos;
- Centralizar as informações operacionais em um banco de dados;
- Controlar o acesso às funcionalidades de acordo com o perfil do usuário;
- Garantir maior organização e integridade dos dados.

---

2 REFERENCIAL TEÓRICO

O desenvolvimento do sistema será baseado em conceitos de Engenharia de Software, levantamento e especificação de requisitos, modelagem de dados, programação orientada a objetos e desenvolvimento de APIs.

A persistência das informações será realizada em um banco de dados relacional utilizando Microsoft SQL Server.

Para a comunicação entre a aplicação e o banco de dados será utilizado o Prisma ORM, responsável pelo mapeamento dos modelos de dados e pela execução das operações de persistência.

O backend será desenvolvido utilizando TypeScript em conjunto com o framework Express.js, responsável pela criação da API e pelo gerenciamento das requisições realizadas pelos clientes da aplicação.

O Docker será utilizado para a criação de ambientes isolados e padronizados para os serviços necessários ao funcionamento do sistema.

---

3 STAKEHOLDERS

Os principais stakeholders do sistema são:

- **Garçom:** responsável pelo atendimento dos clientes e registro dos pedidos.

- **Cozinheiro:** responsável pelo acompanhamento e preparação dos pedidos.
- **Gerente:** responsável pelo gerenciamento e acompanhamento das operações do estabelecimento.
- **Cliente:** pessoa que realiza o pedido e recebe os produtos.
- **Chic Dog:** estabelecimento beneficiado pela organização e controle das operações proporcionados pelo sistema.

---

4 ATORES DO SISTEMA

4.1 Garçom

O garçom utilizará o sistema para:

- Realizar autenticação;
- Registrar pedidos;
- Adicionar itens aos pedidos;
- Alterar pedidos permitidos;
- Cancelar pedidos permitidos;
- Enviar pedidos para preparação;
- Consultar pedidos;
- Acompanhar o status dos pedidos;
- Registrar ou encaminhar informações relacionadas ao pagamento, conforme as permissões definidas.

4.2 Cozinheiro

O cozinheiro utilizará o sistema para:

- Realizar autenticação;
- Visualizar pedidos;
- Consultar detalhes dos pedidos;
- Identificar pedidos pendentes;
- Iniciar a preparação;
- Atualizar o status dos pedidos;
- Indicar a conclusão da preparação.

4.3 Gerente

O gerente utilizará o sistema para:

- Realizar autenticação;
- Gerenciar usuários;
- Consultar pedidos;

- Consultar pagamentos;
- Acompanhar informações operacionais;
- Executar funcionalidades administrativas disponibilizadas pelo sistema.

4.4 Cliente

O cliente será o consumidor dos serviços oferecidos pelo estabelecimento e estará relacionado aos pedidos realizados.

Quando disponibilizado pelo sistema, poderá:

- Solicitar pedidos por meio do atendimento;
- Acompanhar informações do pedido;
- Realizar o pagamento no estabelecimento.

---

5 REQUISITOS FUNCIONAIS

RF01 – Autenticação de usuários

O sistema deverá permitir que usuários cadastrados realizem login utilizando suas credenciais.

RF02 – Controle de acesso

O sistema deverá controlar o acesso às funcionalidades de acordo com o perfil do usuário autenticado.

RF03 – Cadastro de usuários

O sistema deverá permitir o cadastro dos usuários que poderão utilizar o sistema.

RF04 – Gerenciamento de usuários

O sistema deverá permitir que usuários com permissão administrativa consultem e gerenciem os usuários cadastrados.

RF05 – Registro de pedidos

O sistema deverá permitir que o garçom registre novos pedidos para os clientes.

RF06 – Inclusão de itens

O sistema deverá permitir adicionar um ou mais itens a um pedido.

Para cada item deverá ser possível registrar sua quantidade e seu valor correspondente dentro do pedido.

RF07 – Cálculo do valor total

O sistema deverá calcular o valor total de um pedido com base nos itens adicionados, suas quantidades e respectivos valores.

RF08 – Consulta de pedido

O sistema deverá permitir consultar as informações de um pedido, incluindo seus itens, valores e situação atual.

RF09 – Envio do pedido para a cozinha

O sistema deverá disponibilizar o pedido para a cozinha após sua confirmação.

RF10 – Visualização da fila de pedidos

O cozinheiro deverá conseguir visualizar os pedidos que aguardam preparação ou que estejam em processo de preparo.

RF11 – Atualização do status

O sistema deverá permitir a alteração do status dos pedidos durante seu processamento.

RF12 – Alteração de pedido

O sistema deverá permitir a alteração de um pedido enquanto as regras de negócio permitirem sua modificação.

RF13 – Cancelamento de pedido

O sistema deverá permitir o cancelamento de pedidos quando as condições definidas pelas regras de negócio forem atendidas.

RF14 – Registro de pagamento

O sistema deverá permitir registrar o pagamento relacionado a um pedido.

RF15 – Consulta de pagamentos

Usuários autorizados deverão conseguir consultar informações referentes aos pagamentos registrados.

RF16 – Persistência de informações

O sistema deverá armazenar no banco de dados as informações necessárias para o funcionamento da aplicação.

Entre elas estão:

- Usuários;
- Perfis de acesso;
- Credenciais de autenticação;
- Pedidos;
- Itens;
- Itens pertencentes aos pedidos;
- Pagamentos;
- Status dos pedidos.

RF17 – Rastreamento de operações

O sistema deverá registrar informações necessárias para identificar o usuário responsável por operações relevantes realizadas sobre os dados.

RF18 – Consulta de histórico

Usuários autorizados deverão conseguir consultar informações históricas dos pedidos e pagamentos armazenados no sistema.

---

6 REQUISITOS NÃO FUNCIONAIS

RNF01 – Desempenho

O sistema deverá apresentar tempo de resposta adequado para as operações de cadastro, consulta e atualização realizadas normalmente.

RNF02 – Usabilidade

As funcionalidades deverão possuir interface e fluxo de utilização simples e intuitivo para os funcionários do estabelecimento.

RNF03 – Segurança

O sistema deverá utilizar autenticação para restringir o acesso às funcionalidades que exigem identificação do usuário.

RNF04 – Controle de permissões

As operações disponíveis deverão ser determinadas pelo perfil e pelas permissões do usuário autenticado.

RNF05 – Integridade dos dados

O banco de dados deverá utilizar mecanismos de integridade para evitar registros inconsistentes entre usuários, pedidos, itens e pagamentos.

RNF06 – Confiabilidade

As informações persistidas deverão permanecer armazenadas de forma consistente após a conclusão das operações.

RNF07 – Manutenibilidade

A aplicação deverá ser desenvolvida de maneira modular, facilitando a manutenção e evolução futura do sistema.

RNF08 – Escalabilidade

A estrutura do sistema deverá permitir a inclusão futura de novas funcionalidades, usuários e operações sem necessidade de alterações estruturais excessivas.

RNF09 – Compatibilidade

O backend deverá ser desenvolvido utilizando TypeScript e Express.js, com execução sobre Node.js.

RNF10 – Banco de dados

O sistema deverá utilizar Microsoft SQL Server como sistema gerenciador de banco de dados.

RNF11 – ORM

O acesso aos dados deverá ser realizado utilizando Prisma ORM.

RNF12 – Conteinerização

O ambiente do sistema deverá utilizar Docker para facilitar a configuração, execução e padronização dos serviços.

---

7 REGRAS DE NEGÓCIO

RN01 – Autenticação obrigatória

Funcionalidades restritas deverão estar disponíveis somente para usuários autenticados.

RN02 – Perfil de usuário

Cada usuário deverá possuir um perfil que determine suas permissões dentro do sistema.

RN03 – Responsável pelo pedido

Todo pedido deverá possuir um usuário responsável pelo seu registro.

RN04 – Itens do pedido

Um pedido deverá possuir pelo menos um item para ser considerado válido.

RN05 – Quantidade de itens

Cada item pertencente a um pedido deverá possuir uma quantidade válida e maior que zero.

RN06 – Valor do item

Cada item deverá possuir um valor válido para cálculo do total do pedido.

RN07 – Valor total

O valor total do pedido deverá corresponder à soma dos valores de seus itens, considerando suas respectivas quantidades.

RN08 – Status do pedido

Todo pedido deverá possuir um status que represente sua situação atual no fluxo de atendimento.

Os status poderão incluir:

- Pendente;
- Em preparo;
- Pronto;
- Entregue;
- Cancelado.

RN09 – Alteração de pedido

Um pedido poderá ser alterado somente enquanto seu status permitir modificações.

RN10 – Bloqueio durante preparo

Após o início da preparação, o pedido não poderá ser alterado pelo garçom.

RN11 – Cancelamento

O cancelamento de um pedido deverá respeitar seu status atual e as permissões do usuário responsável pela operação.

RN12 – Preparação

O cozinheiro será responsável pela atualização do pedido durante o processo de preparação.

RN13 – Pagamento

Um pagamento deverá estar associado a um pedido existente.

RN14 – Valor do pagamento

O valor registrado no pagamento deverá corresponder ao valor devido pelo pedido, de acordo com as regras definidas para o estabelecimento.

RN15 – Integridade referencial

Não deverá ser possível criar registros que apontem para entidades inexistentes.

Por exemplo:

- Um pedido não poderá apontar para um usuário inexistente;
- Um item de pedido não poderá apontar para um pedido inexistente;
- Um pagamento não poderá apontar para um pedido inexistente.

RN16 – Exclusão de dados relacionados

Registros que possuam dependências ou relacionamentos importantes deverão possuir regras de exclusão que preservem a integridade dos dados históricos.

RN17 – Auditoria

Operações relevantes deverão permitir identificar o usuário responsável pela ação.

---

8 MODELO DE DADOS

O banco de dados deverá representar as principais informações utilizadas pelo sistema.

As entidades deverão ser definidas a partir dos requisitos e posteriormente transformadas em modelos do Prisma.

As principais entidades inicialmente previstas são:

- Usuário;
- Login;
- Garçom;
- Cozinheiro;
- Gerente;
- Cliente;
- Pedido;
- Item;
- ItemPedido;
- Pagamento.

A existência definitiva, os atributos e os relacionamentos dessas entidades deverão ser definidos durante a elaboração do modelo conceitual e do modelo lógico.

---

9 ENTIDADES PREVISTAS

9.1 Usuário

Representa uma pessoa autorizada a utilizar o sistema.

Informações previstas:

- Identificador;
- Nome;
- Perfil;
- Dados necessários para identificação;
- Situação do usuário.

9.2 Login

Representa as informações necessárias para autenticação de um usuário.

Informações previstas:

- Identificador;
- Usuário associado;
- Credencial de acesso;
- Informações relacionadas à autenticação.

As credenciais deverão ser armazenadas de forma segura.

9.3 Garçom

Representa o funcionário responsável pelo atendimento e registro dos pedidos.

Deverá estar associado a um usuário do sistema.

9.4 Cozinheiro

Representa o funcionário responsável pela preparação dos pedidos.

Deverá estar associado a um usuário do sistema.

9.5 Gerente

Representa o funcionário responsável pelas funções administrativas e gerenciais.

Deverá estar associado a um usuário do sistema.

9.6 Cliente

Representa o cliente que realiza o pedido no estabelecimento.

Um cliente poderá possuir um ou mais pedidos registrados no sistema.

9.7 Pedido

Representa uma solicitação realizada por um cliente.

Informações previstas:

- Identificador;
- Data e hora;
- Status;
- Valor total;
- Tempo estimado;
- Cliente;
- Garçom responsável;
- Informações relacionadas à preparação;
- Itens;
- Pagamento.

9.8 Item

Representa um produto que pode ser solicitado pelo cliente.

Informações previstas:

- Identificador;
- Nome;
- Descrição;
- Preço;
- Situação de disponibilidade.

9.9 ItemPedido

Representa a associação entre um pedido e um item.

Informações previstas:

- Identificador;
- Pedido;
- Item;
- Quantidade;
- Preço praticado no momento da venda;
- Subtotal.

A entidade ItemPedido deverá permitir registrar quais produtos pertencem a cada pedido sem duplicar as informações dos produtos.

9.10 Pagamento

Representa o pagamento realizado para um pedido.

Informações previstas:

- Identificador;
- Pedido;
- Valor;
- Forma de pagamento;
- Status;
- Data e hora do pagamento.

---

10 RELACIONAMENTOS PREVISTOS

Os relacionamentos abaixo deverão ser avaliados e formalizados durante a elaboração do modelo conceitual e lógico.

10.1 Usuário – Login

Um usuário deverá possuir suas informações de autenticação associadas.

Cardinalidade prevista:

Usuário 1 : 1 Login

10.2 Usuário – Garçom

Um usuário poderá possuir o perfil de garçom.

Cardinalidade prevista:

Usuário 1 : 0..1 Garçom

10.3 Usuário – Cozinheiro

Um usuário poderá possuir o perfil de cozinheiro.

Cardinalidade prevista:

Usuário 1 : 0..1 Cozinheiro

10.4 Usuário – Gerente

Um usuário poderá possuir o perfil de gerente.

Cardinalidade prevista:

Usuário 1 : 0..1 Gerente

10.5 Cliente – Pedido

Um cliente poderá realizar diversos pedidos.

Cada pedido deverá estar associado a um cliente.

Cardinalidade prevista:

Cliente 1 : N Pedido

10.6 Garçom – Pedido

Um garçom poderá registrar diversos pedidos.

Cada pedido deverá possuir um garçom responsável pelo registro.

Cardinalidade prevista:

Garçom 1 : N Pedido

10.7 Pedido – ItemPedido

Um pedido poderá possuir diversos registros de ItemPedido.

Cada ItemPedido deverá pertencer a um pedido.

Cardinalidade:

Pedido 1 : N ItemPedido

10.8 Item – ItemPedido

Um item poderá estar presente em diversos pedidos.

Cada ItemPedido deverá representar um item específico.

Cardinalidade:

Item 1 : N ItemPedido

Dessa forma, o relacionamento entre Pedido e Item será representado por meio de ItemPedido.

10.9 Pedido – Pagamento

Um pedido poderá possuir um pagamento associado, conforme o fluxo financeiro adotado pelo estabelecimento.

Cardinalidade prevista:

Pedido 1 : 0..1 Pagamento

10.10 Cozinheiro – Pedido

Um cozinheiro poderá atuar na preparação de diversos pedidos.

O pedido poderá registrar o cozinheiro responsável pela preparação, caso essa informação seja necessária no fluxo definido.

Cardinalidade prevista:

Cozinheiro 1 : N Pedido

---

11 DIAGRAMAS UML

11.1 Diagrama de Caso de Uso

O diagrama de caso de uso deverá representar as principais interações entre os atores e o sistema.

Garçom

- Realizar login;
- Criar pedido;
- Adicionar item;
- Alterar pedido;
- Cancelar pedido;
- Enviar pedido para cozinha;
- Consultar pedido;
- Acompanhar status;
- Registrar pagamento, quando autorizado.

Cozinheiro

- Realizar login;
- Visualizar pedidos;
- Visualizar detalhes;
- Iniciar preparo;
- Atualizar status;
- Finalizar preparo.

Gerente

- Realizar login;
- Gerenciar usuários;
- Consultar pedidos;
- Consultar pagamentos;
- Consultar informações administrativas;
- Acompanhar operações.

Cliente

- Realizar pedido por meio do atendimento;
- Acompanhar pedido quando disponibilizado;
- Realizar pagamento no estabelecimento.

---

12 FLUXO PRINCIPAL DO PEDIDO

O fluxo principal esperado para o processamento de um pedido é:

1. O garçom realiza a autenticação no sistema.
2. O sistema valida as credenciais.
3. O garçom inicia um novo pedido.
4. O cliente informa os produtos desejados.
5. O garçom adiciona os itens ao pedido.
6. O sistema registra as quantidades e valores.
7. O sistema calcula o valor total.
8. O garçom confirma o pedido.
9. O sistema registra o pedido no banco de dados.
10. O pedido é disponibilizado para a cozinha.
11. O cozinheiro visualiza o pedido.
12. O cozinheiro inicia a preparação.
13. O status do pedido é atualizado.
14. O cozinheiro finaliza a preparação.
15. O pedido é marcado como pronto.
16. O pedido é entregue ao cliente.

17. O pagamento é registrado.
18. O pedido é finalizado.

---

13 BANCO DE DADOS

13.1 Sistema Gerenciador

O sistema utilizará o **Microsoft SQL Server** como sistema gerenciador de banco de dados.

13.2 ORM

O **Prisma ORM** será utilizado para representar os modelos da aplicação e realizar a comunicação com o SQL Server.

O arquivo `schema.prisma` deverá representar a estrutura definida no modelo lógico do banco.

13.3 Integridade

O banco de dados deverá utilizar:

- Chaves primárias;
- Chaves estrangeiras;
- Restrições de unicidade;
- Restrições de valores obrigatórios;
- Restrições de integridade referencial;
- Tipos de dados adequados;
- Relacionamentos entre entidades.

13.4 Modelagem

A modelagem do banco deverá ser realizada em duas etapas principais:

1. **Modelo conceitual:** identificação das entidades, atributos relevantes e relacionamentos.
2. **Modelo lógico/teórico:** definição das tabelas, chaves primárias, chaves estrangeiras, tipos de dados e restrições.

A implementação física será realizada posteriormente no SQL Server por meio do Prisma.

---

14 API E BACKEND

O backend do sistema será desenvolvido posteriormente utilizando:

- **Node.js:** ambiente de execução;
- **TypeScript:** linguagem de desenvolvimento;
- **Express.js:** framework para construção da API;
- **Prisma:** ORM para comunicação com o banco de dados;
- **SQL Server:** armazenamento persistente dos dados.

A API deverá disponibilizar endpoints responsáveis pelas operações de usuários, autenticação, pedidos, itens e pagamentos.

A implementação da API deverá respeitar os requisitos e regras de negócio estabelecidos neste documento.

---

15 DOCKER

O Docker será utilizado para padronizar o ambiente de desenvolvimento e execução do sistema.

A configuração deverá permitir a execução dos serviços necessários ao projeto de maneira isolada e reproduzível.

Entre os serviços previstos estão:

- Banco de dados SQL Server;
- Backend Express.js;
- Prisma;
- Demais serviços necessários à execução da aplicação.

A configuração definitiva dos containers será estabelecida durante a etapa de implementação.

---

16 CRITÉRIOS DE ACEITAÇÃO

CA01 – Autenticação

O sistema deverá permitir o login de usuários cadastrados utilizando credenciais válidas.

CA02 – Segurança

O sistema deverá impedir o acesso às funcionalidades restritas por usuários não autenticados.

CA03 – Controle de acesso

Um usuário não deverá executar operações que não estejam disponíveis para seu perfil.

CA04 – Registro de pedido

O garçom deverá conseguir criar um pedido e adicionar seus respectivos itens.

CA05 – Cálculo do pedido

O valor total deverá ser calculado corretamente considerando quantidade e preço dos itens.

CA06 – Persistência

Um pedido confirmado deverá ser armazenado no banco de dados.

CA07 – Cozinha

O cozinheiro deverá conseguir visualizar os pedidos destinados à preparação.

CA08 – Status

O sistema deverá permitir a atualização do status do pedido conforme o fluxo operacional.

CA09 – Alteração

O sistema deverá impedir alterações em pedidos que já estejam em preparação.

CA10 – Cancelamento

O sistema deverá respeitar as regras de negócio para permitir ou impedir o cancelamento de pedidos.

CA11 – Pagamento

O sistema deverá permitir registrar um pagamento associado a um pedido existente.

CA12 – Integridade

O banco deverá impedir registros com relacionamentos inválidos.

---

17 RISCOS

R01 – Falha no banco de dados

A indisponibilidade do SQL Server poderá impedir operações de leitura e gravação.

R02 – Falha de comunicação

Problemas de comunicação entre a API, Prisma e SQL Server poderão impedir o processamento das operações.

R03 – Dados inconsistentes

Erros na modelagem ou implementação das relações poderão gerar inconsistências no banco.

R04 – Falhas de autenticação

Problemas no mecanismo de autenticação poderão comprometer o controle de acesso.

R05 – Alterações indevidas

Falhas no controle de permissões poderão permitir operações incompatíveis com o perfil do usuário.

R06 – Erros no cálculo

Problemas no cálculo dos valores poderão gerar divergências entre os itens e o total do pedido.

R07 – Falhas na atualização de status

Uma atualização incorreta poderá comprometer o fluxo entre atendimento, cozinha e entrega.

---

18 CONCLUSÃO

O presente documento estabelece os requisitos para o desenvolvimento do Sistema de Gestão de Pedidos da Chic Dog, contemplando os principais atores, funcionalidades, regras de negócio, requisitos não funcionais e informações que deverão ser gerenciadas pela aplicação.

A especificação foi estruturada considerando a nova abordagem tecnológica do projeto, baseada em SQL Server, Prisma, Express.js, TypeScript e Docker.

A definição das entidades e dos relacionamentos fornece a base necessária para a próxima etapa do projeto, que consiste na elaboração do modelo conceitual e do modelo lógico/teórico do banco de dados.

A partir desses modelos será possível definir as tabelas, chaves primárias, chaves estrangeiras, restrições e demais elementos necessários para a implementação do banco de dados SQL Server.

Posteriormente, o modelo lógico será utilizado como referência para a criação do `schema.prisma`, permitindo que a aplicação desenvolvida em TypeScript e Express.js utilize uma estrutura de dados consistente e alinhada aos requisitos do sistema.

Dessa forma, o documento serve como base para o desenvolvimento incremental do projeto, garantindo que a implementação do banco de dados e das funcionalidades futuras esteja alinhada às necessidades operacionais da Chic Dog.

---

19 AUTORES

Miguel Henrique Isaac de Oliveira

Luca Telini Crozara

Glauber Macário